

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Криптография и сетевая безопасность»

Выполнил:
Студент 4-го курса
Группы АС-66
Головкина В.Д.
Проверил:
Хацкевич А.С.

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 1

Ход работы

Задание 1: В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

Требования:

- * реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- * подобрать параметры генератора (a , c , N , x_0 и др.) так, чтобы добиться максимального периода последовательности $T_{\max}$ (согласно теоретическим условиям из методички);
- * провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

1	Комбинирование алгоритмов генерации методом Макларена – Марсальи	$N=10$ $K=5$
---	---	-----------------

Задание 2: На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту.

Требования:

- * реализовать стандартную схему генерации кандидата в простые числа:
 - * сгенерировать случайное p -битовое число n (используя генератор из Этапа 1);
 - * установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
 - * выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;
- * интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. (Примечание: для метода Михалеску используется собственная специфическая схема генерации доказуемо простых чисел).

1	Тест Соловея – Штрассена
---	--------------------------

Код:

```
from collections import Counter
import random
```

ЧАСТЬ 1. ГЕНЕРАТОР МАКЛАРЕНА — МАРСАЛЬИ (как было)

```
class LCG:
```

```
    def __init__(self, m, a, c, seed):
        self.m, self.a, self.c, self.state = m, a, c, seed % m
```

```
    def next(self):
        self.state = (self.a * self.state + self.c) % self.m
        return self.state
```

```
class MacLarenMarsaglia:
```

```
    def __init__(self, n, k, px, py):
        self.n, self.k = n, k
        self.x = LCG(n, *px)
        self.y = LCG(n, *py)
        self.v = [self.x.next() for _ in range(k)]
```

```
    def next(self):
        y = self.y.next()
        s = y % self.k
        out = self.v[s]
        self.v[s] = self.x.next()
        return out
```

```
    def state(self):
        return (self.x.state, self.y.state, tuple(self.v))
```

```
def find_lcg_parameters(n):
```

```
    params = []
    for a in range(1, n):
        for c in range(1, n):
            for seed in range(n):
                gen = LCG(n, a, c, seed)
                start = gen.state
                period = 0
                for _ in range(n + 1):
                    period += 1
                    if gen.next() == start:
                        break
```

```

        if period == n:
            params.append((a, c, seed))
    return params

def find_best_pair(n, k):
    params = find_lcg_parameters(n)
    print(f"Найдено ЛКГ с периодом {n}: {len(params)} вариантов")
    best_pair, best_period = None, 0
    for px in params:
        for py in params:
            if (px[0], px[1]) == (py[0], py[1]):
                continue
            gen = MacLarenMarsaglia(n, k, px, py)
            seen = {gen.state(): 0}
            step = 0
            while True:
                gen.next()
                step += 1
                st = gen.state()
                if st in seen:
                    period = step - seen[st]
                    break
                seen[st] = step
            if period > best_period:
                best_period, best_pair = period, (px, py)
    return best_pair, best_period

def generate_sequence(length, alphabet, n, k, px, py):
    gen = MacLarenMarsaglia(n, k, px, py)
    L = len(alphabet)
    limit = n - (n % L)
    result = []
    while len(result) < length:
        v = gen.next()
        if v >= limit:
            continue
        result.append(alphabet[v % L])
    return "".join(result)

```

CHI2_005 = {1: 3.841, 2: 5.991, 3: 7.815, 4: 9.488, 5: 11.070,

6: 12.592, 7: 14.067, 8: 15.507, 9: 16.919,
10: 18.307, 11: 19.675, 12: 21.026, 13: 22.362,
14: 23.685, 15: 24.996, 16: 26.296}

```
def chi_square_test(sequence, alphabet, alpha=0.05):
    n, m = len(sequence), len(alphabet)
    expected = n / m
    counts = Counter(sequence)
    chi2 = sum(((counts.get(s, 0) - expected) ** 2) / expected for s in alphabet)
    df = m - 1
    critical = CHI2_005.get(df, df * 1.5)
    print("\n--- Тест  $\chi^2$  ---")
    print(f" $\alpha$  = {alpha}, m = {m}, n = {n}, ожидаемая частота = {expected:.2f}")
    for s in alphabet:
        print(f" '{s}': {counts.get(s, 0)}")
    print(f" $\chi^2$  = {chi2:.4f},  $\chi^2_{\text{крит}}$  = {critical:.4f}, df = {df}")
    print("✓ Равномерность НЕ отвергается." if chi2 < critical
          else "✗ Равномерность отвергается.")
```

```
def serial_test(sequence, alphabet):
    m = len(alphabet)
    pairs = Counter(zip(sequence[:-1], sequence[1:]))
    total = len(sequence) - 1
    expected = total / (m * m)
    chi2 = sum((pairs.get((a, b), 0) - expected) ** 2 / expected
               for a in alphabet for b in alphabet)
    df = m * m - 1
    critical = CHI2_005.get(df, df * 1.5)
    print("\n--- Тест серий (пар) ---")
    print(f"df = {df},  $\chi^2$  = {chi2:.4f},  $\chi^2_{\text{крит}} \approx$  {critical:.4f}")
    print("✓ Пары распределены равномерно." if chi2 < critical
          else "✗ Есть зависимость между соседними символами.")
```

```
def find_period(s):
    n = len(s)
    for period in range(1, n // 2 + 1):
        for start in range(0, n // 2):
            ok = True
            for i in range(start, n - period):
```

```

        if s[i] != s[i + period]:
            ok = False
            break
    if ok:
        return period, start
    return None, None

```

ЧАСТЬ 2. ГЕНЕРАЦИЯ ПРОСТОГО ЧИСЛА

Список малых простых из задания (все < 256)

```

SMALL_PRIMES = [
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53,
    59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113,
    127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181,
    191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251
]

```

```

def is_divisible_by_small_primes(n):
    """Шаг 3: проверка делимости на малые простые."""
    if n in SMALL_PRIMES:
        return False
    for p in SMALL_PRIMES:
        if n % p == 0:
            return True
    return False

```

```

def jacobi(a, n):
    """
    Символ Якоби (a/n).
    n — нечётное положительное.
    """
    if n <= 0 or n % 2 == 0:
        raise ValueError("n должно быть положительным нечётным")
    a = a % n
    result = 1
    while a != 0:
        while a % 2 == 0:
            a //= 2
        if n % 8 in (3, 5):
            result = -result

```

```

a, n = n, a
if a % 4 == 3 and n % 4 == 3:
    result = -result
a = a % n
return result if n == 1 else 0

```

```
def solovay_strassen(n, k=5):
```

```
    """
```

Тест Соловея — Штрассена.

Возвращает True, если n вероятно простое.

k — число раундов (в задании k = 5).

```
    """
```

```
    if n < 2:
```

```
        return False
```

```
    if n == 2 or n == 3:
```

```
        return True
```

```
    if n % 2 == 0:
```

```
        return False
```

```
    for _ in range(k):
```

```
        a = random.randint(2, n - 2)
```

```
        x = jacobi(a, n)
```

```
        if x == 0:
```

```
            return False # НОД(a,n) > 1
```

```
        # Сравнение Эйлера:  $a^{(n-1)/2} \equiv (a/n) \pmod{n}$ 
```

```
        if pow(a, (n - 1) // 2, n) != x % n:
```

```
            return False
```

```
    return True
```

```
def generate_prime(bits):
```

```
    """
```

Генерация простого числа по схеме задания:

1) случайное p-битное n,

2) установить старший и младший биты в 1,

3) проверка делимости на малые простые,

4) 5 тестов Соловея — Штрассена.

Используется перебор последовательных нечётных чисел.

```
    """
```

```
    # Шаг 1: случайное p-битное число
```

```
    n = random.getrandbits(bits)
```

```
# Шаг 2: старший бит = 1 (гарантия длины), младший бит = 1 (нечётность)
n |= (1 << (bits - 1)) # старший бит
n |= 1                 # младший бит
```

```
attempts = 0
while True:
    attempts += 1
    # Шаг 3: проверка делимости на малые простые
    if not is_divisible_by_small_primes(n):
        # Шаг 4: 5 раундов Соловея — Штрассена
        if solovay_strassen(n, k=5):
            return n, attempts
    # Переход к следующему нечётному числу
    n += 2
```

```
def find_prime_between_limits():
    """Альтернативный режим: поиск простого в заданном диапазоне."""
    print("\n(Перебор последовательных нечётных чисел)")
    return generate_prime(int(input("Введите битность p: ")))[0]
```

ЧАСТЬ 3. МЕНЮ

```
def task1_maclaren():
    N, K = 10, 5
    print("=" * 55)
    print(f"Задача 1: Макларен — Марсалья, N={N}, K={K}")
    print("=" * 55)

    alphabet = input("Введите алфавит без пробелов: ").strip()
    if len(alphabet) < 2 or len(alphabet) > N:
        print(f"Алфавит должен содержать 2..{N} символов.")
        return

    try:
        length = int(input("Введите длину последовательности: "))
    except ValueError:
        print("Длина должна быть целым числом.")
        return

    if length <= 0:
        print("Длина должна быть > 0.")
        return
```

```

print("\nПоиск параметров X и Y...")
(px, py), max_period = find_best_pair(N, K)
print(f"X: a={px[0]}, c={px[1]}, seed={px[2]}")
print(f"Y: a={py[0]}, c={py[1]}, seed={py[2]}")
print(f"Максимальный период комбинации: {max_period}")

seq = generate_sequence(length, alphabet, N, K, px, py)
print(f"\nПервые 100 символов:\n{seq[:100]}")

p, start = find_period(seq)
if p:
    print(f"\nПериод выходной строки: {p} (начиная с позиции {start})")
else:
    print("\nПериод не найден в пределах длины.")

chi_square_test(seq, alphabet)
serial_test(seq, alphabet)

```

```

def task2_prime():
    print("=" * 55)
    print("Задача 2: Генерация простого числа (Соловей — Штрассен)")
    print("=" * 55)

```

```

try:
    bits = int(input("Введите битность p (например, 16, 32, 64): "))
except ValueError:
    print("Битность должна быть целым числом.")
    return
if bits < 4:
    print("Минимальная битность — 4.")
    return

```

```

print(f"\nГенерация {bits}-битного простого числа...")
prime, attempts = generate_prime(bits)

```

```

print(f"\n✓ Найдено простое число:")
print(f"  n = {prime}")
print(f"  Битов: {prime.bit_length()}")

```

```

print(f" Попыток перебора: {attempts}")
print(f" Двоичное представление: {bin(prime)}")

# Продемонстрируем, что тест Соловея — Штрассена проходит 5 раз
print(f"\nПрогон 5 тестов Соловея — Штрассена для найденного числа:")
for i in range(1, 6):
    a = random.randint(2, prime - 2)
    x = jacobi(a, prime)
    ok = pow(a, (prime - 1) // 2, prime) == x % prime
    print(f" Тест {i}: a = {a}, Якоби = {x}, "
          f"a^((n-1)/2) mod n = {pow(a, (prime-1)//2, prime)} "
          f"→ {'✓' if ok else 'X'}")

```

```

def main():
    while True:
        print("\n" + "=" * 55)
        print("МЕНЮ")
        print("=" * 55)
        print("1 — Задача 1: Макларен — Марсалья")
        print("2 — Задача 2: Генерация простого числа")
        print("0 — Выход")

```

```

choice = input("Выбор: ").strip()

```

```

if choice == "1":
    task1_maclaren()
elif choice == "2":
    task2_prime()
elif choice == "0":
    print("Выход.")
    break
else:
    print("Неверный выбор.")

```

```

if __name__ == "__main__":
    main()

```

```

=====
МЕНЮ
=====
1 – Задача 1: Макларен – Марсалья
2 – Задача 2: Генерация простого числа
0 – Выход
Выбор: 1
=====
Задача 1: Макларен – Марсалья, N=10, K=5
=====
Введите алфавит без пробелов: ABGWIUUD
Введите длину последовательности: 1000

Поиск параметров X и Y...
Найдено ЛКГ с периодом 10: 40 вариантов
X: a=1, c=1, seed=0
Y: a=1, c=3, seed=0
Максимальный период комбинации: 10

Первые 100 символов:
IGUWBDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABGWIUUDABG

Период выходной строки: 7 (начиная с позиции 5)

--- Тест  $\chi^2$  ---
 $\alpha = 0.05$ , m = 7, n = 1000, ожидаемая частота = 142.86
'A': 142
'B': 143
'G': 143
'W': 143
'I': 143

```

$\chi^2 = 0.0060$, $\chi^2_{\text{крит}} = 12.5920$, df = 6

✓ Равномерность НЕ отвергается.

--- Тест серий (пар) ---

df = 48, $\chi^2 = 5924.4204$, $\chi^2_{\text{крит}} \approx 72.0000$

✗ Есть зависимость между соседними символами.

```

=====
МЕНЮ
=====

```

```

1 – Задача 1: Макларен – Марсалья
2 – Задача 2: Генерация простого числа
0 – Выход

```

Выбор: 2

```

=====
Задача 2: Генерация простого числа (Соловей – Штрассен)
=====

```

Введите битность p (например, 16, 32, 64): 64

Генерация 64-битного простого числа...

✓ Найдено простое число:

n = 9326427266849565251

Битов: 64

Попыток перебора: 7

Двоичное представление: 0b1000000101101110001000000010111100010000001010111101001000011

Прогон 5 тестов Соловея – Штрассена для найденного числа:

Тест 1: a = 8570593639408456222, Якоби = -1, $a^{((n-1)/2)} \bmod n = 9326427266849565250 \rightarrow \checkmark$

Тест 2: a = 5817872145312815760, Якоби = 1, $a^{((n-1)/2)} \bmod n = 1 \rightarrow \checkmark$

Тест 3: a = 2839041502958317220, Якоби = -1, $a^{((n-1)/2)} \bmod n = 9326427266849565250 \rightarrow \checkmark$

Тест 4: a = 5706657607079898703, Якоби = -1, $a^{((n-1)/2)} \bmod n = 9326427266849565250 \rightarrow \checkmark$

Вывод: в ходе лабораторной работы я освоила программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.